

Chapter 57: Enterprise Scenarios

Scenario 1: Micro-frontend vs. Monolithic Shell for a 40-Team Org

Situation: Your company has 40 teams contributing to a single Angular application that has grown to a 25-minute production build and a shared `main.ts` that nobody fully understands anymore. Deploys are coordinated on a weekly train because any team's change can break another team's feature. Leadership wants independent deploys per team by next quarter.

Question: How would you evaluate whether to break this into micro-frontends (e.g., using Module Federation) versus investing in a better-modularized monolith with Nx, and what would your migration plan look like?

Answer: Start by diagnosing *why* deploys are coupled — it's rarely just "one repo." Usually it's shared global state, a shared design-system library that isn't versioned independently, and a build graph with no boundaries. Before reaching for micro-frontends (which trade build-time coupling for runtime coupling and operational complexity — version skew between remotes, shared dependency duplication, harder E2E testing, cross-remote routing/state), I'd first try an Nx-based modular monolith:

```
// nx.json - enforce module boundaries via lint rules
{
  "targetDefaults": {
    "build": { "cache": true, "dependsOn": ["^build"] }
  }
}
```

```
// eslint rule enforcing team ownership boundaries
{
  "rules": {
    "@nx/enforce-module-boundaries": ["error", {
      "depConstraints": [
        { "sourceTag": "scope:billing", "onlyDependOnLibsWithTags": ["scope:billing", "scope:shared"] },
        { "sourceTag": "scope:orders", "onlyDependOnLibsWithTags": ["scope:orders", "scope:shared"] }
      ]
    }]
  }
}
```

This gets you affected-based CI (`nx affected --target=build`), independently testable feature libs, and — critically — you can still ship a single deployable artifact while dramatically cutting build time (25 min → often 3-5 min with distributed caching/Nx Cloud). Only if teams genuinely need **independent runtime deploys** (different release cadences, different tech stacks, contractual isolation between business units) would I escalate to Module Federation, and even then I'd federate at 3-5 "domain" boundaries, not 40 team boundaries — federation overhead per remote is real (duplicate Angular runtime unless you dedupe via `shared` config, network waterfall for remote entry points, harder debugging).

My recommended path: (1) introduce Nx and tag-based boundaries, (2) measure actual coupling via `nx graph`, (3) extract 3-5 true bounded contexts as remotes only where independent deploy cadence is a hard business requirement, (4) keep everything else as fast, cached, boundary-enforced libs in the monolith. This is cheaper, keeps a single version of Angular/RxJS in memory, and solves the *actual* problem (slow, coupled CI) without inheriting distributed-systems problems in the frontend.

Interviewer intent: Tests whether the candidate reaches for a trendy architecture pattern reflexively or diagnoses the root cause and picks the cheapest solution that satisfies the real constraint.

Scenario 2: A Shared UI Library Breaking Change Across 12 Consuming Apps

Situation: Your team owns `@acme/ui-kit`, a shared Angular component library consumed by 12 internal applications maintained by different teams. You need to change the `AcmeButtonComponent` API — the `variant` input is being split into `variant` and `tone` to support a new design system, and the old single-input API is semantically ambiguous going forward.

Question: How do you roll out this breaking change without a "big bang" migration that blocks 12 teams simultaneously?

Answer: Never ship a breaking change as a single major version bump with a changelog note and hope. The plan:

1. **Additive first, breaking never (until forced):** Add the new `tone` input alongside the existing `variant`, with `tone` defaulting to a value derived from `variant` internally, so existing consumers see zero behavior change.

```
@Component({
  selector: 'acme-button',
  standalone: true,
  template: `<button [class]="computedClass()"><ng-content /></button>`
})
export class AcmeButtonComponent {
  variant = input<'primary' | 'secondary' | 'danger'>('primary');
  tone = input<'brand' | 'neutral' | 'critical' | undefined>(undefined);

  private resolvedTone = computed(() =>
    this.tone() ?? { primary: 'brand', secondary: 'neutral', danger: 'critical' }
    [this.variant()]
  );

  computedClass = computed(() => `btn btn-${this.resolvedTone()}`);
}
```

2. **Deprecate loudly but non-breakingly:** Mark `variant` `@deprecated` in TSDoc so it surfaces in every consuming team's IDE, and log a dev-mode console warning (stripped in prod builds via `ngDevMode`) when `variant` is used without `tone`.
3. **Provide a codemod.** Ship an `ng update @acme/ui-kit` schematic that rewrites `[variant]='danger'` to `[tone]='critical'` automatically across consumer templates using the

Angular schematics `Tree` API — this is the single biggest lever for a library maintainer's credibility; teams will not manually touch 12 codebases.

4. **Version and support window:** Publish this as a minor version (non-breaking). Only in the *next major* (announced with a 1-2 sprint lead time and tracked in a shared migration doc) do you actually remove `variant`. Use semantic versioning strictly and communicate via a changelog + a Slack/Teams channel dedicated to design-system consumers, not just a changelog nobody reads.
5. **Contract tests:** Maintain a small consumer-driven contract test suite (or at minimum a "golden" Storybook snapshot per consuming team's critical usage patterns) so you catch when a "non-breaking" change actually breaks visual/behavioral contracts before publishing.

The core principle: breaking changes in shared libraries are a *process* problem, not a code problem. The code (additive API, deprecation, codemod) is what makes the process survivable.

Interviewer intent: Evaluates experience owning a shared library consumed by many teams — specifically whether the candidate knows about schematics-based codemods and additive migration, not just "bump the major version."

Scenario 3: Design System Governance — Who Owns What

Situation: Three different teams have each built their own `DatePicker` component because the central design-system team's version didn't support a feature they needed, and raising a request took three weeks to get prioritized. Now there are three inconsistent date pickers in production, and a new VP of Product is asking why the app "doesn't feel like one product."

Question: As the staff engineer asked to fix this, what governance model and technical structure would you propose?

Answer: This is a governance failure disguised as a technical one — teams didn't build duplicates because they wanted to, they built duplicates because the contribution path to the shared library was slower than shipping their own. The fix has to address both the process and the code:

Process — federated contribution model: Move from a "central team gatekeeps everything" model to a "central team owns the API contract and quality bar, feature teams can contribute directly" model, similar to how large open-source design systems (e.g., Spotify's Backstage, Shopify's Polaris) operate. Concretely:

- A lightweight RFC template for new component APIs (1-page, not a 3-week review cycle).
- A "component council" with rotating representation from consuming teams, meeting biweekly, empowered to approve additive changes in the meeting itself.
- An SLA: any request for a new prop/slot gets a decision (not necessarily implementation) within 3 business days.

Technical — make extension the path of least resistance instead of forking:

```
// Design the DatePicker to be extensible via content projection and injection
// instead of forcing every variant into the core team's roadmap.
@Component({
  selector: 'acme-date-picker',
  standalone: true,
  template: `
    <div class="picker-shell">
```

```

    <ng-content select="[pickerHeader]" />
    <acme-calendar [config]="mergedConfig()" />
    <ng-content select="[pickerFooter]" />
  </div>
  ,
})
export class AcmeDatePickerComponent {
  config = input<Partial<DatePickerConfig>>({});
  private defaults = inject<DATE_PICKER_DEFAULTS>();
  mergedConfig = computed(() => ({ ...this.defaults, ...this.config() }));
}

// Teams can override behavior via DI without forking the component
export const DATE_PICKER_DEFAULTS = new InjectionToken<DatePickerConfig>
('DATE_PICKER_DEFAULTS', {
  factory: () => ({ minDate: null, maxDate: null, weekStartsOn: 0 })
});

```

Consolidation plan: Audit the three existing date pickers, identify the union of requirements (this is usually where the "why did you fork" answer lives — e.g., one team needed a range picker, another needed fiscal-year support). Fold those into the canonical component's config surface, then deprecate the forks with the same additive/codemod approach as Scenario 2. Track adoption with a lint rule banning direct imports of the forked components org-wide once the canonical one is ready.

The deeper lesson for the VP conversation: consistency is a *process SLA* problem as much as a component API problem. I'd present both fixes together, because fixing only the code without fixing the 3-week review bottleneck guarantees the next fork.

Interviewer intent: Probes whether the candidate can operate at a socio-technical level — recognizing that governance/process failures often masquerade as technical debt.

Scenario 4: Migrating 200 Components from NgModules to Standalone Without a Big-Bang Rewrite

Situation: You have a 6-year-old Angular application with roughly 200 components across 40 NgModules, actively developed by 8 teams. Leadership wants to move to the standalone APIs to reduce boilerplate and unlock esbuild/Vite-based builds, but a stop-the-world rewrite is not acceptable — feature work cannot pause.

Question: Walk through your incremental migration strategy, including how standalone and NgModule-based code coexist safely during the transition.

Answer: Angular explicitly supports bidirectional interop, which is what makes this safe to do incrementally:

1. **Tooling first:** Run `ng generate @angular/core:standalone` — Angular's official schematic does this in three passes: (a) convert components/directives/pipes to standalone, (b) remove now-unnecessary NgModule declarations, (c) prune empty modules. Run it module-by-module, not repo-wide, so each PR is reviewable and team-scoped.
2. **Interop during transition:**

```
// A still-NgModule-based feature can import a standalone component directly
@NgModule({
  declarations: [LegacyDashboardComponent],
  imports: [StandaloneWidgetComponent], // standalone components go in `imports`, not `declarations`
})
export class LegacyDashboardModule {}
```

```
// A standalone component can still pull in an NgModule that hasn't migrated yet
@Component({
  standalone: true,
  imports: [LegacyChartsModule, NewStandaloneCardComponent],
  template: `...`
})
export class DashboardShellComponent {}
```

3. **Sequencing by risk, not by ease:** Migrate leaf components first (no downstream consumers), then shared/common modules, then routed feature modules last, because routed modules touch `RouterModule.forChild` and lazy-loading config, which is the highest-risk part (`loadChildren` → `loadComponent`).

```
// Before: NgModule-based lazy route
{ path: 'billing', loadChildren: () => import('./billing/billing.module').then(m => m.BillingModule) }

// After: standalone route with functional providers scoped to the route
{
  path: 'billing',
  loadComponent: () => import('./billing/billing-shell.component').then(m => m.BillingShellComponent),
  providers: [provideBillingFeature()]
}
```

4. **Guardrails:** Add an ESLint rule (or a custom `nx affected` check) that bans new NgModules from being created going forward, so the ratio only moves in one direction while migration is in progress.
5. **Bootstrap last:** Only flip `bootstrapModule` → `bootstrapApplication` once all root-level dependencies (router, http, animations) have functional provider equivalents in place, since that's the point of no return for the whole app.
6. **Sequence with the team's calendar:** Assign each of the 40 modules to its owning team as a "boy scout rule" task attached to unrelated feature PRs where possible, plus 1-2 dedicated migration sprints for the gnarliest shared modules (routing, core, shared-ui). This avoids a dedicated multi-month migration team while still making steady progress.

The key message: this is a strangler-fig migration, not a rewrite — interop makes every intermediate state shippable.

Interviewer intent: Distinguishes candidates who've actually run a large real migration from those who only know the individual APIs; tests sequencing judgment (leaf-first, routes-last) and awareness of interop mechanics.

Scenario 5: Onboarding a New Senior Engineer to a 500k-LOC Codebase

Situation: You just hired a senior engineer with strong Angular experience from a smaller company. Two weeks in, they're still struggling to make changes confidently because the codebase has years of accumulated patterns: some feature modules use NgRx, some use plain services with `BehaviorSubject`, some use new signal-based state, routing is split across 6 different `app-routing` files, and there's no single source of truth for "how do we do X here."

Question: What structural and process investments would you make to fix onboarding, and how do you decide whether to consolidate the mixed state-management patterns or leave them alone?

Answer: This is fundamentally a documentation-debt and architecture-decision-record (ADR) problem compounding with genuine incidental complexity. My approach:

Immediate (days):

- A living `ARCHITECTURE.md` (or an internal docs site) with a decision map: "if your feature needs cross-page shared state → use signal store pattern X; if it's local UI state → use component signals; NgRx is legacy-only, do not adopt for new features." This single artifact turns "tribal knowledge" into a lookup table.
- A "golden path" example feature — one fully-migrated, idiomatic feature module new engineers can clone as a template, annotated with comments explaining *why*, not just *what*.

Structural (weeks):

- Standardize routing: consolidate to a single top-level route config using `provideRouter` with feature routes lazy-loaded via `LoadChildren / LoadComponent`, even if the *internals* of each feature aren't migrated yet. New engineers need one place to trace "how did I get to this URL," even if the destination code is still legacy.
- Introduce a state-management decision tree as an actual ADR, and only migrate *opportunistically* — do not do a NgRx-to-signals rewrite as a dedicated project unless there's a concrete pain point (e.g., NgRx boilerplate is measurably slowing feature velocity, or bundle size from `@ngrx/store` + `@ngrx/effects` is a real cost). Consolidating three patterns into one is valuable, but a forced migration for consistency alone risks becoming a multi-quarter distraction with no user-facing value. My rule of thumb: migrate a module's state management only when you're already touching that module for a feature reason.

Process:

- Pair the new engineer with a "code archaeologist" session in week 1 — a senior engineer walks through the git history/blame of the 2-3 messiest shared files, explaining the historical context (this is often faster than any doc for transferring tribal knowledge).
- Track onboarding friction explicitly: have new hires log every "I didn't know X" moment in their first month into a shared doc; triage that list quarterly — it's your most honest signal of where documentation/consistency debt actually costs velocity, versus where it's cosmetic.

Interviewer intent: Tests pragmatism around technical debt — recognizing that not all inconsistency needs fixing, and that onboarding pain is a legitimate, measurable signal for prioritizing which debt to pay down first.

Scenario 6: A Shared HTTP Interceptor Silently Breaking One Team's Feature

Situation: A platform team added a global HTTP interceptor that adds a correlation ID header and retries failed requests up to twice for observability/resilience reasons. Three weeks later, the Payments team reports that duplicate payment submissions are occurring in production — the retry logic is re-submitting non-idempotent POST requests.

Question: How do you fix this at the architecture level so a shared cross-cutting concern like this can never silently break a consuming team's feature again?

Answer: The immediate fix is straightforward — retries should never apply to non-idempotent verbs by default:

```
export const retryInterceptor: HttpInterceptorFn = (req, next) => {
  const isIdempotent = req.method === 'GET' || req.method === 'HEAD' || req.method === 'PUT';
  if (!isIdempotent) {
    return next(req); // no retry for POST/PATCH/DELETE by default
  }
  return next(req).pipe(
    retry({ count: 2, delay: 300 })
  );
};
```

But the real fix is architectural — this incident happened because a **global, silent, cross-cutting behavior change** was applied without an opt-in/opt-out contract or idempotency awareness. To prevent recurrence:

1. Explicit opt-in per request via context tokens, not implicit global behavior for mutating verbs:

```
export const RETRYABLE = new HttpContextToken<boolean>(() => false);

// Payments team explicitly declares their POST is NOT safe to retry (default already false)
this.http.post(url, body, { context: new HttpContext().set(RETRYABLE, false) });

// A team with a genuinely idempotent POST (e.g., idempotency-key-backed) opts in explicitly
this.http.post(url, body, { context: new HttpContext().set(RETRYABLE, true) });
```

```
export const retryInterceptor: HttpInterceptorFn = (req, next) => {
  if (!req.context.get(RETRYABLE)) return next(req);
  return next(req).pipe(retry({ count: 2, delay: 300 }));
};
```

- 2. Change review process for global interceptors/guards:** Any change to a globally-registered interceptor, guard, or functional provider that affects *all* HTTP traffic or *all* routes needs a cross-team RFC and a canary rollout (feature-flagged, or rolled out to one app/team first), not a direct merge — because its blast radius is the entire app, unlike a normal feature PR.
- 3. Contract tests for critical financial flows:** The Payments team should have an integration test asserting "a POST to `/payments` is called at most once given a simulated network failure," which would have caught this in CI before production, regardless of what the interceptor did.

4. **Observability:** Ensure the correlation ID (which the platform team added for good reason) is logged alongside retry attempts, so the next incident like this is diagnosed in minutes via logs instead of days of production investigation.

Interviewer intent: Tests whether the candidate treats globally-scoped cross-cutting code (interceptors, guards) with the elevated caution their blast radius demands, and knows `HttpContextToken` as the idiomatic mechanism for per-request opt-in/opt-out.

Scenario 7: Bundle Size Regression Caught Only in Production

Situation: Your CI pipeline doesn't fail builds on bundle size. A recent PR added a `moment.js`-based date formatting utility to a shared `core` library used by every lazy-loaded feature, and it wasn't caught until a customer complained about slow load times. Analysis shows initial bundle grew by 180KB gzipped because the utility was imported in an eagerly-loaded shared module.

Question: What controls would you put in place — both to prevent this specific class of regression and to catch it automatically in CI going forward?

Answer: Multiple layers, because no single control catches everything:

1. Automated budget enforcement in CI, not just `angular.json` warnings:

```
// angular.json – budgets should fail the build, not just warn, for initial bundle
"budgets": [
  { "type": "initial", "maximumWarning": "500kb", "maximumError": "700kb" },
  { "type": "anyComponentStyle", "maximumWarning": "6kb", "maximumError": "10kb" }
]
```

`maximumError` actually fails `ng build` — many teams only set `maximumWarning`, which CI ignores.

2. Bundle-diff bot on every PR: Use `source-map-explorer` or a bundle-analyzer step in CI that comments on the PR with the delta versus `main`, so a 180KB regression is visible to the reviewer *before* merge, not after a customer complaint:

```
# CI step (conceptual)
- run: npx nx build core --stats-json
- run: npx bundlewatch --config bundlewatch.config.json
```

3. Dependency hygiene rules for shared/core libraries specifically: A library that's imported eagerly by every feature deserves a stricter budget and stricter dependency review than a leaf feature module. I'd add an explicit rule: any new third-party dependency added to `libs/core` or `libs/shared-ui` requires a comment justifying bundle cost and confirming tree-shakability, enforced via PR template checklist plus the automated diff bot.

4. Fix the specific regression correctly: Don't just remove `moment.js` reactively — replace it with a tree-shakable, lightweight alternative (`date-fns` with named imports, or the `Intl.DateTimeFormat` / `Intl.RelativeTimeFormat` APIs already in the browser, avoiding a date library dependency entirely for simple formatting):


```
// Native Intl instead of a 70KB+ library for common cases
const formatted = new Intl.DateTimeFormat('en-US', { dateStyle: 'medium' }).format(date);
```

5. Lazy-load even shared utilities where possible: If a feature only occasionally needs heavy formatting logic, defer it:

```
@defer (on interaction) {
  <app-rich-date-editor [locale]="locale()" />
} @placeholder {
  <span>{{ date() | date }}</span>
}
```

The systemic fix is making bundle size a CI-enforced, PR-visible metric — the same rigor as test coverage — rather than something discovered via a customer complaint weeks later.

Interviewer intent: Tests operational maturity around performance regressions — whether budgets are just configured or actually enforced, and awareness of `@defer` and tree-shakable alternatives as concrete mitigations.

Scenario 8: Coordinating a Breaking Angular Major Version Upgrade Across 15 Repos

Situation: Your organization has 15 separate Angular application repositories, all depending on a shared `@acme/core` npm package. Angular v20 introduces a breaking change relevant to your codebase (e.g., a removed deprecated API you rely on), and you're the platform engineer responsible for coordinating the upgrade.

Question: Describe your upgrade strategy — order of operations, how you avoid blocking 15 teams simultaneously, and how you validate the upgrade didn't silently break anything.

Answer: Treat this like a dependency-graph problem, not 15 independent upgrades:

1. Upgrade the leaf dependency first: `@acme/core` has no dependents within the org other than the 15 apps, so it can be upgraded, tested, and published independently. Run `ng update @angular/core @angular/cli` on it first, using `ng update`'s automatic migration schematics to handle most mechanical changes.

```
ng update @angular/core@20 @angular/cli@20 --create-commits
```

Publish `@acme/core` as a new major version (since dropping support for old Angular is itself a breaking change for consumers), with a clear compatibility matrix in the changelog: `@acme/core v5.x → Angular 20, v4.x → Angular 17-19`.

2. Sequence the 15 app upgrades by risk and blast radius, not alphabetically:

- Upgrade 1-2 low-traffic internal tools first as canaries — this surfaces migration issues cheaply.
- Then the next tier of apps, in parallel across teams (each team owns their own app's upgrade — this doesn't have to be serialized since `@acme/core` is already stable).
- Customer-facing, high-revenue apps go last, once the schematic/migration playbook is proven.

3. Give every team a concrete playbook, not just "go upgrade": a checklist covering the specific breaking API removed, a `grep` command to find usages in their repo, and the codemod/replacement snippet, e.g.:

```
// If the breaking change removed `ViewChild` static resolution ambiguity or similar:  
// before  
@ViewChild(ChildComponent) child: ChildComponent;  
// after (explicit static flag now required/changed)  
@ViewChild(ChildComponent, { static: false }) child?: ChildComponent;
```

4. Automated validation, not manual QA per app: Each app's existing CI (unit + e2e) is the safety net — but I'd also add a smoke-test gate specifically targeting the breaking change's blast radius (e.g., if the break was in forms validation timing, add a targeted test asserting validators still fire when expected).

5. Track progress centrally: A shared dashboard (even a simple spreadsheet or a Nx-style project graph annotation) showing each of the 15 repos' current Angular version and `@acme/core` version, so the platform team can see stragglers and unblock them rather than discovering 6 months later that 3 apps are still on the old major and now blocking a security patch.

6. Deprecation deadline with teeth: Give a firm date after which `@acme/core` v4.x stops receiving security patches, escalated to engineering leadership if a team hasn't started by the midpoint of the window — upgrades without a deadline tend to never finish.

Interviewer intent: Tests large-scale rollout coordination skills — sequencing by risk, providing tooling/playbooks rather than just mandates, and having an accountability mechanism so the migration actually completes.

Scenario 9: Zoneless Change Detection Migration for a Legacy-Heavy App

Situation: Leadership wants to adopt zoneless Angular (`provideZonelessChangeDetection()`) for performance reasons across a large app. The codebase has many third-party jQuery-based widgets wrapped in Angular components, and several places where state is mutated outside Angular's normal input/signal flow (e.g., direct DOM manipulation callbacks from a charting library updating a plain class field read in the template).

Question: What risks does zoneless change detection introduce here, and how would you de-risk the migration?

Answer: Zone.js today patches async APIs (`setTimeout`, DOM events, promises) and triggers change detection automatically after any of them fire. Removing Zone.js means Angular only re-renders when it's explicitly told to — via signals, `markForCheck()` / `ChangeDetectorRef`, or `AsyncPipe`. The core risk here is exactly the "class field mutated by a jQuery callback, read directly in the template" pattern — with zone.js, a stray DOM event elsewhere would incidentally trigger CD and the template would happen to update; without zone.js, nothing tells Angular to re-render, and the UI goes stale silently.

De-risking plan:

- 1. Audit for "invisible" state updates first.** Grep for patterns where component class fields are assigned inside non-Angular callbacks (jQuery `.on()`, third-party SDK callbacks, raw `addEventListener` outside `HostListener`) and are read in templates without an explicit CD trigger.

2. Convert audited state to signals — this is the actual fix, not a workaround:

```
// Before: silently relies on zone.js picking up the jquery callback
export class ChartWrapperComponent implements AfterViewInit {
  selectedPoint: DataPoint | null = null; // read in template – breaks zoneless

  ngAfterViewInit() {
    $(this.el.nativeElement).on('click', (e, point) => {
      this.selectedPoint = point; // zone.js used to save us here
    });
  }
}

// After: signal-based, explicit, zoneless-safe
export class ChartWrapperComponent implements AfterViewInit {
  selectedPoint = signal<DataPoint | null>(null);

  ngAfterViewInit() {
    $(this.el.nativeElement).on('click', (e, point) => {
      this.selectedPoint.set(point); // signal write schedules CD regardless of zone
    });
  }
}
```

3. For code you can't immediately refactor, use `NgZone.run()` as an explicit, temporary bridge rather than assuming implicit pickup — a clearly marked shim so it's greppable and removable later:

```
constructor(private zone: NgZone) {}
ngAfterViewInit() {
  $(this.el.nativeElement).on('click', (e, point) => {
    this.zone.run(() => { this.selectedPoint = point; }); // TODO(zoneless-migration):
    // remove once signal-based
  });
}
```

4. **Incremental rollout, not a flag flip:** Roll out zoneless behind a build flag on an internal/staging environment first, run the full E2E suite (which is your best signal for "silent staleness" bugs, since they often manifest as "button click had no visible effect"), and dogfood internally for a sprint before production.
5. **Guard third-party wrapper components specifically:** Since jQuery widgets are the highest-risk surface, prioritize auditing and converting exactly those wrapper components first, since they're a small, identifiable set compared to auditing the entire app.

The overarching message to leadership: zoneless is a real perf win (no more spurious CD passes from unrelated timers/events) but it's not a config flip in a codebase with legacy imperative DOM code — it requires converting "implicit" state mutation to "explicit" signal-based mutation wherever zone.js was silently compensating.

Interviewer intent: Tests deep understanding of what zone.js actually does (schedules CD after async APIs) versus surface-level "just add the flag," and whether the candidate can identify concrete legacy patterns that break under zoneless.

Scenario 10: API Contract Drift Between Frontend and 6 Backend Microservices

Situation: Your Angular app consumes 6 different backend microservices, each owned by a different backend team, each with its own OpenAPI spec that changes independently. Twice in the last quarter, a backend team renamed a field or changed a response shape without notifying frontend, causing production runtime errors that TypeScript's compile-time checking didn't catch because the DTOs were hand-written and outdated.

Question: How do you build a system that catches API contract drift before it reaches production, given that hand-maintained TypeScript interfaces clearly aren't enough?

Answer: Hand-written interfaces are a false sense of safety — they document what you *think* the API looks like at the moment you wrote them, with no mechanism to detect drift. The fix is to make the contract executable and continuously verified:

1. Generate types from the OpenAPI spec instead of hand-writing them, so the frontend's types are always derived from the actual contract, not a stale guess:

```
npx openapi-typescript https://backend-team/orders/openapi.json -o libs/api-types/orders.d.ts
```

Run this generation step in CI on a schedule (or triggered by the backend team's spec-publish webhook) and fail the build if generated types differ from committed types without a corresponding PR — this alone surfaces "backend changed something and nobody told us" immediately instead of at runtime.

2. Runtime validation at the boundary, since compile-time types don't protect you from an actual malformed response in production:

```
import { z } from 'zod';

const OrderSchema = z.object({
  orderId: z.string(),
  status: z.enum(['pending', 'shipped', 'delivered']),
  total: z.number(),
});
type Order = z.infer<typeof OrderSchema>;

@Injectable({ providedIn: 'root' })
export class OrdersService {
  private http = inject(HttpClient);

  getOrder(id: string) {
    return this.http.get(`/api/orders/${id}`).pipe(
      map(raw => OrderSchema.parse(raw)) // throws loudly instead of silently propagating
      `undefined`
    );
  }
}
```

This converts "field renamed, `order.total` is now `undefined`, and some downstream calculation silently produces `NaN` in production" into an immediate, loud, alertable error at the exact API boundary, with a clear stack trace.

3. Consumer-driven contract tests (e.g., Pact) between frontend and each backend team, so a backend team's CI fails *before they merge* if their change breaks a contract the frontend depends on — this is strictly better than "frontend catches it after deploy" because it shifts the fix left to the team that caused it.

4. A shared API registry/changelog with mandatory notification, even a simple one: backend teams publish spec changes to a shared channel/registry, and the OpenAPI-diff CI job (point 1) posts automatically, removing reliance on someone remembering to "notify frontend."

5. Prioritize by blast radius: Apply strict schema validation first to the highest-traffic/highest-revenue-impact services (checkout, payments), and roll out to the rest incrementally, since retrofitting all 6 services with zod schemas is real effort and shouldn't block on doing all of them at once.

Interviewer intent: Tests whether the candidate understands the difference between compile-time type safety (which only reflects what a human wrote) and runtime contract verification (which reflects reality), and knows concrete tools (openapi-typescript, zod, Pact) rather than just saying "better communication."

Scenario 11: Feature Flag Sprawl Causing Untestable Combinatorial State

Situation: Over two years, your app has accumulated 40 feature flags controlling UI behavior, many of which interact (flag A changes a component's behavior only when flag B is also on). QA reports they can't realistically test all combinations, and a bug was recently found in production that only manifested with a specific 3-flag combination that had never been tested.

Question: How do you bring this under control architecturally, without simply telling QA to "test more combinations"?

Answer: Combinatorial flag explosion is a design smell — it means flags are being used as a permanent architecture mechanism instead of a temporary rollout mechanism. The fix has two parts: reduce the *number of live combinations* and make the remaining ones *structurally impossible to interact accidentally*.

1. Classify every flag by intended lifetime and retire aggressively:

- *Release toggles* (rolling out a feature) — should be deleted within weeks of 100% rollout. Audit for flags at 100%/0% for 90+ days and delete them; this alone often cuts 40 flags to 15-20.
- *Ops toggles* (kill switches) — legitimately permanent, but should be few and isolated (e.g., "disable third-party widget X if it's down"), not entangled with UI logic branches.
- *Experiment toggles* (A/B tests) — have a hard expiration date tied to the experiment's end date, enforced by a linter/CI check that fails the build if an experiment flag is referenced past its declared expiry.

2. Architecturally isolate flags so they can't silently interact:

```
// Anti-pattern: flags checked ad-hoc, scattered, and combinable in unpredictable ways
if (flags.newCheckout() && flags.expressShipping() && !flags.legacyDiscount()) { ... }

// Better: a single resolved "feature configuration" computed once per session,
// so combination logic lives in ONE tested place instead of scattered across components
export const checkoutConfig = computed(() => resolveCheckoutConfig(flagsSignal()));

function resolveCheckoutConfig(flags: FlagState): CheckoutConfig {
  // all flag-interaction logic centralized and unit-testable in isolation
  if (flags.newCheckout && flags.expressShipping) return EXPRESS_NEW_CONFIG;
  if (flags.newCheckout) return STANDARD_NEW_CONFIG;
  return LEGACY_CONFIG;
}
```

Centralizing resolution means the *set of reachable configurations* is finite and enumerable (you can unit test `resolveCheckoutConfig` against every input combination cheaply, since it's a pure function), instead of combinations emerging implicitly from scattered `if` checks across components.

3. Pairwise/combinatorial testing for the flags that must remain, rather than exhaustive testing — tools like pairwise test-case generators cover most real-world interaction bugs (which are usually pairwise, not triple-or-higher) with a fraction of the test cases of full combinatorial coverage.

4. Governance: Require an expiry date and an owning team as mandatory metadata when creating any new flag, surfaced on a dashboard, with automated Slack reminders to the owning team as expiry approaches. Flags without an assigned owner get auto-flagged for removal review.

The architectural point for the interview: the fix isn't "test harder," it's "reduce the reachable state space" via lifecycle discipline and centralizing interaction logic into pure, testable functions instead of scattered conditionals.

Interviewer intent: Tests whether the candidate sees feature flags as a lifecycle-managed engineering artifact (with debt implications) rather than a free, permanent mechanism, and can propose a concrete architectural pattern (centralized resolution) to tame combinatorial complexity.

Scenario 12: Enforcing a Consistent Error-Handling Contract Across 8 Teams

Situation: Different teams handle HTTP errors inconsistently: some show a raw error object in a toast, some redirect to a generic error page for any 4xx/5xx, some silently swallow errors, and one team's feature crashes the whole app on an unhandled `RxJS` error because they forgot a `catchError` in a critical stream. A recent production incident occurred because a 401 from an expired token wasn't handled by one team's module, and rather than triggering a re-login flow, it just left a frozen loading spinner.

Question: How do you design a consistent, enterprise-wide error handling strategy that doesn't rely on every individual developer remembering to add `catchError` everywhere?

Answer: Individual developer discipline doesn't scale across 8 teams — the strategy needs to make the correct behavior the default, with escape hatches for legitimate exceptions, rather than relying on every stream having a manually-added `catchError`.

1. Centralize categorical error handling in an interceptor so it doesn't depend on per-call discipline:

```

export const errorInterceptor: HttpInterceptorFn = (req, next) => {
  const auth = inject(AuthService);
  const notifications = inject(NotificationService);
  const router = inject(Router);

  return next(req).pipe(
    catchError((err: HttpResponse) => {
      if (err.status === 401) {
        auth.triggerReAuth(); // centrally guarantees this NEVER depends on a feature team remembering it
        return EMPTY;
      }
      if (err.status === 403) {
        router.navigate(['/forbidden']);
        return EMPTY;
      }
      if (err.status >= 500) {
        notifications.showError('Something went wrong. Please try again.');
```

2. A global `ErrorHandler` as the last line of defense for anything that still slips through (unhandled RxJS errors, template errors, etc.), so a single team's bug degrades gracefully instead of white-screening the whole app:

```

@Injectable()
export class AppErrorHandler implements ErrorHandler {
  private logger = inject(LoggingService);

  handleError(error: unknown): void {
    this.logger.logToRemote(error); // observability first, always
    // Never let this re-throw and crash the whole app; that's the entire point of a global handler
  }
}
```

```

// bootstrap
bootstrapApplication(AppComponent, {
  providers: [{ provide: ErrorHandler, useClass: AppErrorHandler }]
});
```

3. A shared "result" pattern for feature-level error UI, so features have a consistent, typed way to represent success/error/loading instead of inventing their own per team:

```

export type RemoteData<T> =
  | { status: 'loading' }
  | { status: 'success'; data: T }
```

```
| { status: 'error'; error: string };
```

```
// In a component using signals + toSignal
data = toSignal(
  this.service.getData().pipe(
    map(data => ({ status: 'success', data } as const)),
    catchError(err => of({ status: 'error', error: mapErrorMessage(err) } as const)),
    startWith({ status: 'loading' } as const)
  ),
  { initialValue: { status: 'loading' } as const }
);
```

4. Enforce via lint + code review checklist, not just documentation: an ESLint rule flags any `.subscribe()` call chain missing an error callback or a preceding `catchError`, and a PR template checklist item asks "does this stream have explicit error handling or rely on the global interceptor/handler?"

5. Postmortem-driven governance: the frozen-spinner 401 incident should result in an actual regression test (simulate an expired token, assert re-auth flow triggers) added to the shared testing utilities so *every* team's HTTP-consuming feature gets this coverage for free via a shared test harness, not by each team remembering to write it.

Interviewer intent: Tests systems thinking around making "the right thing" the path of least resistance (interceptor + global handler + shared result type) instead of relying on decentralized developer discipline across many teams.

Scenario 13: Choosing Between NgRx, Signal Store, and Plain Services for a New Greenfield Module

Situation: You're kicking off a brand-new, fairly complex feature module (multi-step wizard with cross-step validation, undo/redo, and real-time collaboration via WebSocket updates from multiple users editing the same document) inside an existing large app that already has NgRx set up for other features.

Question: Would you use NgRx, `@ngrx/signals` (Signal Store), or plain services with signals for this new module, and how do you justify that decision to the team given the existing NgRx investment?

Answer: This isn't a "pick your favorite" decision — it should be driven by the specific requirements of *this* feature, weighed against consistency costs of introducing a third pattern.

Requirements analysis:

- Undo/redo strongly favors an approach with clear, serializable state transitions — this is NgRx's classic strength (the reducer pattern naturally gives you a state history you can snapshot/replay).
- Real-time collaborative updates from a WebSocket need a clean way to merge external events into state predictably — again, an action/reducer-style flow makes "external WebSocket event → dispatch action → reducer merges" traceable and testable, versus scattered signal `.set()` calls from multiple event sources which get harder to reason about as merge conflicts multiply.
- Cross-step wizard validation benefits from centralized, inspectable state (Redux DevTools time-travel debugging for a customer-reported "the wizard got into a weird state" bug is genuinely valuable here).

Given that, I'd lean toward **NgRx** for this specific module, *not because it's already used elsewhere*, but because undo/redo + collaborative merge + time-travel debugging are exactly NgRx's home turf. I'd resist the urge to force-fit `@ngrx/signals` or plain services just because they're newer — newness isn't the deciding factor, requirements fit is.

```
// Reducer-driven undo/redo is natural here: each action is a state transition you can snapshot
export const wizardReducer = createReducer(
  initialWizardState,
  on(WizardActions.stepCompleted, (state, { stepId, data }) => ({
    ...state,
    history: [...state.history, state.present], // undo stack
    present: { ...state.present, steps: { ...state.present.steps, [stepId]: data } },
  })),
  on(WizardActions.undo, (state) => ({
    ...state,
    present: state.history.at(-1) ?? state.present,
    history: state.history.slice(0, -1),
  })),
);
```

However, I'd push back if the team's instinct was "use NgRx because the rest of the app does" for a *simpler* module — for a basic CRUD settings page with no undo/redo/collaboration complexity elsewhere in the app, plain signals or `@ngrx/signals` would be far less boilerplate, and I'd advocate for that instead, even inside an NgRx-heavy app. Consistency matters, but not at the cost of forcing every module into a pattern mismatched to its actual complexity — the cost of NgRx's ceremony for a trivial feature is real developer friction and slower onboarding for that specific module.

The way I'd sell this to the team: "we already use NgRx elsewhere, and this module's requirements — undo/redo, DevTools time-travel, collaborative merge conflict resolution — are the exact reasons NgRx exists, so we're not introducing inconsistency for its own sake, we're using the right tool that also happens to match our existing investment."

Interviewer intent: Tests whether the candidate picks state management tools based on actual technical requirements (undo/redo, time-travel debugging, event merging) rather than dogma or trend-following in either direction.

Scenario 14: A Cypress/Playwright E2E Suite Taking 3 Hours and Blocking Every Deploy

Situation: Your CI pipeline runs a 3-hour end-to-end test suite (600+ tests) before every production deploy, owned collectively (i.e., owned by nobody) across many teams. Deploys are now happening once every 2 days instead of the desired multiple-times-a-day cadence, and flaky tests routinely require re-runs, adding hours more.

Question: How do you restructure the testing strategy to enable fast, confident, frequent deploys without lowering quality bars?

Answer: A single monolithic 3-hour E2E gate in front of every deploy is usually evidence of an inverted test pyramid — too much confidence riding on slow, flaky, expensive E2E tests instead of fast, reliable unit/integration tests catching most regressions earlier.

1. Parallelize and shard first (quick win): 600 tests split across 20 parallel runners (via Cypress's `--parallel` with a dashboard, or Playwright's built-in sharding) turns 3 hours into ~10-15 minutes wall-clock, at the cost of CI runner capacity — often the fastest lever with no test-quality tradeoff.

```
# playwright shard example
- run: npx playwright test --shard=${{ matrix.shard }}/8
  strategy:
    matrix:
      shard: [1, 2, 3, 4, 5, 6, 7, 8]
```

2. Quarantine flaky tests aggressively and separately from the release gate, tracked with an SLA for the owning team to fix — a flaky test blocking deploys teaches engineers to ignore CI failures ("just re-run it"), which is far more dangerous than the flaky test itself, since it erodes trust in the entire signal.

```
// Explicitly quarantined, tracked, and time-boxed – not silently ignored
test.fixme('collaborative editing sync – flaky, tracked in JIRA-4821', async ({ page }) => {
  ... });
```

3. Re-balance the test pyramid. Push coverage down: business logic validation, form validation rules, state transitions, and API contract shapes should be unit/component tests (`TestBed` / Angular Testing Library, milliseconds each), not full-browser E2E (seconds each). Reserve E2E for genuinely cross-system, critical-path flows: login, checkout, the 5-10 "if this breaks, the business loses money" journeys — not exhaustive coverage of every edge case.

4. Split the gate: fast pre-merge, full suite post-merge/scheduled. Run a curated "smoke" subset (the critical-path 30-50 tests, <10 min) as the pre-deploy gate, and run the full 600-test suite on a schedule (e.g., nightly) or post-deploy against a canary, alerting the owning team if something fails rather than blocking that day's deploys retroactively.

5. Ownership, not "owned by everyone": Assign each E2E test file to an owning team (matching feature ownership), so flaky-test triage doesn't fall to whoever happens to notice CI is red — this alone usually fixes the "nobody fixes flaky tests" problem within a sprint or two once accountability is explicit.

6. Track a "confidence per minute of CI" metric, not just pass/fail — if a 3-hour suite and a re-balanced 15-minute smoke-gate + component-test suite catch the same regressions in practice (validated by injecting known bugs and confirming both catch them, or by historical incident post-mortems), the business case for the faster gate is concrete, not just theoretical.

Interviewer intent: Tests understanding of the test pyramid in practice, ownership/accountability for flaky tests, and pragmatic CI/CD throughput optimization rather than "just add more E2E tests" or "just delete tests" reflexes.

Scenario 15: Handling a Multi-Tenant White-Label Product with Per-Client Theming and Feature Sets

Situation: Your Angular app is sold to 50+ enterprise clients, each with custom branding (colors, logos, fonts) and slightly different enabled feature sets (some clients get an "Advanced Reporting" module, others don't). Currently this is handled via a sprawling number of `*ngIf/@if` checks against a `client.config` object scattered through templates, and adding a new client requires a code change and redeploy.

Question: How would you re-architect this for scalability — so onboarding client #51 doesn't require an engineer to touch component code?

Answer: Scattered `@if` checks against a config object is the classic sign that tenant configuration needs to be pulled out of component logic entirely and into data-driven composition. The redesign:

1. Theming: CSS custom properties resolved at runtime, not compile-time SCSS variables per client.

```
@Injectable({ providedIn: 'root' })
export class ThemeService {
  applyTheme(theme: TenantTheme) {
    const root = document.documentElement.style;
    root.setProperty('--color-primary', theme.primaryColor);
    root.setProperty('--color-secondary', theme.secondaryColor);
    root.setProperty('--font-family', theme.fontFamily);
  }
}
```

This means onboarding a new client's branding is a *data* change (a theme JSON object fetched from a tenant-config service at bootstrap) not a *code* change, and definitely not a redeploy.

2. Feature entitlements: a capability-based model, resolved once, consumed everywhere via signals/DI — not scattered `@if` checks re-deriving the same logic in 40 templates:

```
export const TENANT_CAPABILITIES = new InjectionToken<Signal<TenantCapabilities>>(
  'TENANT_CAPABILITIES');

// A structural directive makes the intent explicit and centralizes the check
@Directive({ selector: '[requiresCapability]', standalone: true })
export class RequiresCapabilityDirective {
  private capabilities = inject(TENANT_CAPABILITIES);
  private tpl = inject(TemplateRef);
  private vcr = inject(ViewContainerRef);

  requiresCapability = input.required<keyof TenantCapabilities>();

  constructor() {
    effect(() => {
      this.vcr.clear();
      if (this.capabilities()[this.requiresCapability()]) {
        this.vcr.createEmbeddedView(this.tpl);
      }
    });
  }
}
```

```
}
```

```
<app-advanced-reporting *requiresCapability="'advancedReporting'" />
```

This reads as intent ("this requires a capability") rather than an opaque config-object equality check, and centralizes the capability-resolution logic in one directive instead of duplicating

`client.config.features.advancedReporting` checks everywhere.

3. Route-level entitlement gating for entire modules a client doesn't have access to, via a functional guard, so disabled features aren't even lazy-loaded for tenants without them:

```
export const capabilityGuard: CanActivateFn = (route) => {  
  const capabilities = inject(TENANT_CAPABILITIES)();  
  const required = route.data['requiredCapability'] as keyof TenantCapabilities;  
  return capabilities[required] ?? false;  
};
```

4. Tenant config as a bootstrap-time resolved value, not scattered runtime lookups: fetch the tenant's theme + entitlements once via an `APP_INITIALIZER`-equivalent (a `provideAppInitializer` in modern Angular) before the app renders, so every downstream component/directive/guard reads from an already-resolved signal rather than each doing its own async lookup.

5. Onboarding client #51 becomes: add a row to a tenant-config database/service (branding JSON + entitlement flags), zero code changes, zero redeploy — which is the actual business goal driving this refactor.

The key architectural shift being tested: moving from "code branches per client" to "data-driven capability resolution," which is the standard pattern for scaling multi-tenant SaaS products.

Interviewer intent: Tests whether the candidate can generalize a scattered conditional-logic problem into a proper capability/entitlement architecture, and knows concrete Angular mechanisms (structural directives, functional guards, DI tokens) to implement it cleanly.

Scenario 16: Cross-Team Disagreement on Lazy-Loading Boundaries Causing Duplicate Chunk Bloat

Situation: A bundle analysis reveals that a shared date-formatting utility, a chart library, and a validation helper are each being duplicated across 8 different lazy-loaded chunks, adding significant redundant weight to the total downloaded JS across a typical user session, even though each chunk individually is under budget.

Question: Why does this happen even when each team followed lazy-loading best practices individually, and how do you fix it at a build-configuration level rather than asking every team to manually coordinate imports?

Answer: This happens because lazy-loaded chunks are, by default, self-contained bundles of everything they import that isn't already in a shared/common chunk — if 8 feature modules each independently import the same chart library, and the bundler's default chunking heuristic doesn't recognize the cross-chunk duplication opportunity (or it does, but the common-chunk threshold config doesn't catch it), you get exactly this: individually-reasonable, collectively-wasteful.

Fix at the build level (this should not be a per-team discipline problem):

1. Tune the bundler's common-chunk extraction. Angular's modern build (`esbuild`-based `application` builder) and the underlying bundler support common-chunk splitting; ensure shared dependencies used by 2+ lazy chunks get automatically hoisted into a shared vendor chunk rather than duplicated. This is often a matter of confirming the shared utility is imported from a single library entry point (so the bundler can dedupe it) rather than being duplicated as separate small utility files across features.

2. Force explicit shared-chunk boundaries via a "shared runtime" library:

```
// libs/shared-runtime – explicitly hoisted, imported by all features consistently
export { formatDate } from './date-utils';
export { ChartwrapperComponent } from './chart-wrapper.component';
```

If every feature imports the chart library *through* this single shared entry point rather than importing the third-party package directly in each feature, the bundler reliably recognizes it as one shared dependency graph node instead of 8 separate ones.

3. Nx's module boundary + dependency graph tooling makes this visible, not implicit: `nx graph` and `nx build --stats-json` surface exactly which libraries are pulled into which chunks, so instead of "8 teams manually coordinating imports," a platform-owned lint rule can enforce "third-party charting/date libraries may only be imported via `@acme/shared-runtime`, never directly," turning a coordination problem into a mechanically-enforced constraint.

4. Measure the *actual* fix with real-user session weight, not just per-chunk budgets. Per-chunk bundle budgets (Scenario 7) don't catch this class of problem, because each chunk individually passes budget — you need a session-weighted metric (e.g., "total unique JS downloaded across the top 5 most common user navigation paths") to see the real redundant cost, and to validate the fix actually reduced it.

5. Consider `@defer` with `prefetch` for genuinely shared-but-heavy dependencies (e.g., the charting library) so it's fetched once, opportunistically, ahead of when most users will need it, rather than being bundled redundantly into each feature that happens to use it:

```
@defer (on viewport; prefetch on idle) {
  <app-chart-wrapper [data]="data()" />
}
```

The core lesson: individually-correct lazy-loading decisions can still produce collectively wasteful bundles, and the fix is a platform-level shared-entry-point convention plus tooling-enforced boundaries — not asking 8 teams to manually notice and coordinate.

Interviewer intent: Tests deeper bundler/build understanding beyond "just lazy load everything" — recognizing emergent duplication across independently-reasonable decisions and fixing it structurally.

Scenario 17: Accessibility (a11y) Debt Discovered Right Before an Enterprise Compliance Deadline

Situation: Your company just signed a major enterprise client whose contract requires WCAG 2.1 AA compliance within 60 days. An audit reveals hundreds of violations across a large app built over 4 years by teams with no consistent a11y practice: missing form labels, color-contrast failures from the design system's palette, custom dropdown components with no keyboard navigation, and modals that don't trap focus.

Question: With a hard 60-day deadline and a large surface area, how do you triage and execute this so you're not just doing a superficial pass?

Answer: With a hard deadline and broad debt, the strategy is risk-weighted triage plus systemic fixes at the component-library level (since most violations trace back to a small number of shared components, not 200 independent bugs).

1. Triage by severity and reach, not by file count. WCAG violations aren't equal: a color-contrast failure in the design system's button component affects every screen using that button (huge reach, systemic fix); a missing `alt` text on one marketing page image is isolated (low reach). Prioritize:

- Design-system-level fixes first (contrast tokens, focus-visible styles, form control labeling patterns) — fixing `@acme/ui-kit` once fixes it everywhere it's used.
- Then interaction-pattern fixes with high reach: the custom dropdown and modal focus-trap issues, since these patterns likely repeat across many features built from the same (broken) base component.
- Long-tail, page-specific issues last, and potentially out of scope for the 60-day deadline if truly low-risk/low-traffic — communicate this tradeoff explicitly to the client-facing team rather than pretending everything can be fixed.

2. Fix the systemic component issues first, since they have the highest leverage:

```
// Focus trap as a reusable directive – fixed once, applied to every modal across the app
@Directive({ selector: '[focusTrap]', standalone: true })
export class FocusTrapDirective implements AfterViewInit, OnDestroy {
  private el = inject(ElementRef<HTMLElement>);
  private previouslyFocused: HTMLElement | null = null;

  ngAfterViewInit() {
    this.previouslyFocused = document.activeElement as HTMLElement;
    const focusable = this.el.nativeElement.querySelectorAll<HTMLElement>(
      'a[href], button:not([disabled]), input, select, textarea,
      [tabindex]:not([tabindex="-1"])'
    );
    focusable[0]?.focus();
    this.el.nativeElement.addEventListener('keydown', this.trapKeydown);
  }

  private trapKeydown = (e: KeyboardEvent) => { /* Tab/Shift+Tab wraparound logic */ };

  ngOnDestroy() {
    this.previouslyFocused?.focus(); // restore focus on close – commonly missed
  }
}
```

```
// Contrast fixed at the design-token level, not per-component
:root {
  --color-text-on-primary: #ffffff; /* re-audited to meet 4.5:1 against --color-primary */
}
```

3. Automated regression prevention so this debt doesn't silently reaccumulate: integrate `axe-core` (via `@axe-core/playwright` or similar) into CI, failing builds on new violations in touched components, so the 60-day fix isn't undone by the next feature PR.

```
test('dashboard has no detectable a11y violations', async ({ page }) => {
  await page.goto('/dashboard');
  const results = await new AxeBuilder({ page }).analyze();
  expect(results.violations).toEqual([]);
});
```

4. Manual audit for what automated tools can't catch: `axe-core` typically catches ~30-50% of WCAG issues (contrast, missing labels, ARIA misuse) but keyboard navigation flows and screen-reader announcement quality need manual testing (NVDA/VoiceOver) — budget real QA time for this, don't rely solely on automated scans as "done."

5. Communicate scope transparently to stakeholders: present a realistic plan — "systemic component fixes close ~70% of violations by reach within 3 weeks; remaining long-tail page-specific issues need an additional X weeks post-deadline, with a documented remediation plan we can share with the client's compliance team" — rather than overpromising 100% in 60 days and quietly cutting corners.

Interviewer intent: Tests whether the candidate treats a11y as a systemic, component-library-level concern (highest leverage) versus a page-by-page checklist, and can manage a compressed-timeline compliance project honestly.

Scenario 18: A Shared Library Team Wants to Force Everyone onto a New Signal-Based API, Feature Teams Are Resisting

Situation: The platform team has built a new signal-based state library intended to replace an older RxJS-`BehaviorSubject`-based shared state service used by every feature team. Feature teams are pushing back — they have working code, a looming quarterly deadline, and see no immediate value in the migration since "it still works."

Question: As the platform lead proposing this, how do you get buy-in rather than mandate it top-down, and how do you sequence the actual migration?

Answer: Mandating a migration that feature teams see no value in — especially against a deadline — breeds resentment and low-quality, box-checking migrations. The better path is proving value concretely and making adoption low-friction, before asking for org-wide commitment.

1. Quantify the actual cost of the status quo, not just "the new thing is nicer." Concrete numbers land better than architecture aesthetics: e.g., "the current `BehaviorSubject` service pattern has caused 4 production bugs this year from stale unsubscribed state / manual `distinctUntilChanged` mistakes," or "onboarding surveys show new engineers take 2 extra days to understand the imperative subscription-management pattern versus signals' automatic cleanup." If the honest answer is "there's no measurable cost, it just feels dated," that's a signal the migration may not be worth prioritizing yet — I'd push back on my own team's plan in that case rather than mandate it anyway.

2. Make interop trivial so migration is low-risk and incremental, not a rewrite:


```
// Bridge: expose the same data as both an Observable (legacy consumers) and a Signal (new consumers)
// so teams can migrate on their own schedule without a flag-day cutover
@Injectables({ providedIn: 'root' })
export class UserStateService {
  private state = signal<UserState>(initialState);

  readonly usersignal = this.state.asReadonly();
  readonly user$ = toObservable(this.state); // legacy RxJS consumers keep working unchanged

  updateUser(partial: Partial<UserState>) {
    this.state.update(s => ({ ...s, ...partial }));
  }
}
```

3. Prove it on the platform team's own code first ("eat your own dog food"), then pilot with one willing feature team, publish the before/after (lines of code, bug reduction, dev-experience feedback) internally, and let the results — not a mandate — drive the next teams to opt in.

4. Decouple migration timing from the deadline entirely. Explicitly tell feature teams: "no one needs to migrate before the quarterly deadline; this is available when you have room, and we'll support both patterns via the interop bridge for at least N quarters." Removing time pressure from something that offers no immediate business value defuses most resistance — the resistance was largely about deadline risk, not the technical merits.

5. Only after adoption reaches a critical mass organically (say, 60-70% of feature teams having migrated because they saw value) would I set a deprecation date for the old pattern, communicated with the same additive/codemod approach as Scenario 2 — by then it's "help us finish something clearly working" rather than "adopt this because platform said so."

The interview-worthy point: technical leads under-invest in the *change management* half of a migration and over-invest in the code; a technically excellent library with a top-down mandate against a resistant, deadline-pressured org usually produces worse outcomes than a good-enough library rolled out with genuine buy-in.

Interviewer intent: Tests leadership/influence skills for a staff-level engineer — whether they understand that cross-team technical adoption requires demonstrated value and low-friction migration paths, not authority-based mandates.

Scenario 19: Build Times Scaling Badly as the Nx Monorepo Grows Past 100 Libraries

Situation: Your Nx monorepo has grown to over 100 libraries across a dozen applications. CI build+test time for a typical PR has crept from 8 minutes to 35 minutes over the past year, even with Nx's affected-command and local caching enabled, because PRs touching a foundational shared library (like `libs/shared-ui` or `libs/core-models`) trigger a rebuild of dozens of downstream dependents.

Question: What levers do you have to bring this back under control, beyond "just enable caching" (which is already on)?

Answer: Local caching solves the "rebuild the same thing twice" problem but doesn't solve "one PR legitimately affects 60 downstream libraries" — that's a dependency-graph shape problem, and a distribution/execution problem.

1. Remote/distributed caching (Nx Cloud or self-hosted equivalent) if not already in place — local cache only helps the same machine/agent; a shared remote cache means if *any* CI run (or any developer) already built a given library at a given input hash, every other CI run gets it for free. This is usually the single biggest lever after local caching is exhausted, and it directly attacks the "60 downstream dependents rebuild" cost since most of those dependents' *other* inputs haven't changed.

2. Distributed Task Execution across multiple CI agents for the affected graph, so the 60 affected projects build in parallel across N machines instead of serially on one — turning a linear scaling problem into a roughly $N/\text{agent-count}$ one.

3. Re-examine why a change to `libs/core-models` affects 60 dependents — that's often a dependency-graph design smell, not an inherent cost. If `core-models` is a single library containing every shared type/interface across the whole org, *any* change to *any* type invalidates everything that imports *anything* from it, even if a given consumer only used an unrelated type. Splitting an overly broad shared library into more granular libraries (by actual bounded context, e.g., `core-models-billing`, `core-models-orders`) means a billing-only change no longer invalidates the orders team's build.

```
// Before: one giant lib, changes to any type ripple to all 60 consumers
libs/core-models/ (billing types + order types + user types + ...)

// After: granular libs, changes are scoped to actual consumers
libs/core-models-billing/
libs/core-models-orders/
libs/core-models-user/
```

4. Tag and lint-enforce boundaries so the graph doesn't silently regrow overly-broad shared libraries (the same `depConstraints` mechanism from Scenario 1), preventing the next `core-models`-style bottleneck from forming.

5. Split CI into a fast pre-merge gate (affected unit tests + lint + typecheck for the actual diff) versus a fuller nightly/scheduled build for anything not on the critical merge path, similar to the E2E restructuring in Scenario 14 — not every one of the 60 affected projects necessarily needs a full rebuild+e2e-test gate on every PR if the change is additive/low-risk (though this needs care — for truly breaking changes to a foundational lib, you *do* want full affected coverage; this is a judgment call based on the nature of the specific PR, potentially automated via risk heuristics like "does this PR only add exports, or does it change existing signatures").

6. Measure and report the actual bottleneck before optimizing blindly — `nx build --graph` and Nx Cloud's run analytics show exactly which projects/tasks dominate wall-clock time; optimize the top 3-5 offenders rather than guessing.

Interviewer intent: Tests build-system depth beyond "enable caching" — understanding remote caching vs. local caching, distributed task execution, and recognizing that dependency-graph shape (overly broad shared libraries) is often the root cause of cascading rebuild cost.

Scenario 20: Reconciling Conflicting Requirements — Product Wants Instant Feature Delivery, Architecture Wants a Refactor First

Situation: Product management wants a major new feature (a real-time notifications center) delivered in 3 weeks. Your team knows the current notification-related code is tightly coupled to a single legacy `NotificationBannerComponent` with no extensibility, and building the new feature "properly" would require roughly 2 weeks of refactoring first, leaving only 1 week for the actual feature — which the team believes is not enough, risking either the deadline or a rushed, bug-prone feature.

Question: As the tech lead, how do you navigate this conflict between business timeline pressure and technical reality, and what would you actually propose?

Answer: This is a negotiation and communication problem as much as a technical one — the wrong move is either silently absorbing the pressure (over-promising 3 weeks and burning the team out or shipping broken code) or unilaterally deciding to refactor first without product's buy-in (which erodes trust when the "invisible" 2 weeks produce no visible feature progress).

1. Make the tradeoff concrete and visible, not abstract. Instead of saying "the code is bad, we need to refactor," quantify it in product-understandable terms: "building the notifications center on the current coupled component means every new notification type (which you've told me are coming in the next two quarters — SMS alerts, in-app banners, email digests) will each take an extra 3-4 days of rework versus 1 day if we invest in extensibility now. The refactor pays for itself by the second new notification type." This reframes it from "engineering wants to gold-plate" to "here's the recurring cost of not doing it."

2. Propose a middle path — a minimal, targeted decoupling, not the full "proper" refactor. Often the full 2-week refactor is doing more than what's strictly needed to unblock this feature; scope down to exactly what the notifications center needs:

```
// Minimal extensibility seam: extract just enough interface to unblock the new feature,  
// without a full redesign of the entire legacy notification system  
export interface NotificationRenderer {  
  canRender(notification: Notification): boolean;  
  render(notification: Notification): TemplateRef<unknown>;  
}  
  
// Legacy banner keeps working entirely unchanged  
export class LegacyBannerRenderer implements NotificationRenderer { /* wraps existing  
component as-is */ }  
  
// New notification center registers its own renderer without touching legacy code  
export class NotificationCenterRenderer implements NotificationRenderer { /* new  
implementation */ }
```

This kind of seam (strangler-fig pattern applied to a single component) often takes 3-4 days, not 2 weeks, because you're not migrating the legacy component's existing behavior, just making the *new* feature pluggable alongside it.

3. Present product with real options and real tradeoffs, and let them make the business call with full information — that's their job, not engineering's to decide unilaterally:

- Option A: 3-4 day minimal decoupling + ~10 days for the feature, roughly matching the 3-week ask with a small buffer, using the seam pattern above.
- Option B: full 3-week feature on the current coupled component, explicitly flagged as accumulating debt that will cost N extra days on each of the next two known notification types.
- Option C: extend the deadline by a few days for a more complete refactor, if product decides the extensibility is worth the delay given the known upcoming roadmap.

4. Whatever is chosen, write it down as an explicit, dated decision (a lightweight ADR: "we chose Option A on [date], accepting X known limitation, revisit when Y happens") so it's a deliberate tradeoff the team and product both own, not a silent shortcut that gets blamed on engineering later when the SMS-alerts feature turns out to be painful.

The staff-level signal here: don't fight product on timeline in the abstract — translate technical debt into concrete, roadmap-relevant cost, offer a scoped middle option instead of "all or nothing," and make the final tradeoff an explicit, shared decision rather than something engineering unilaterally imposes or silently absorbs.

Interviewer intent: Tests stakeholder communication and negotiation skill — whether the candidate can translate technical debt into business terms, propose pragmatic middle-ground scope, and treat the tradeoff as a shared decision rather than a unilateral engineering or product call.

Scenario 21: Legacy AngularJS-to-Angular Coexistence During a Multi-Year Migration

Situation: Your organization still has a substantial AngularJS (1.x) application that has been migrating to modern Angular for three years using `@angular/upgrade`, running both frameworks side-by-side in the same page. Progress has stalled — only 40% of routes have migrated, hybrid-mode overhead (both framework runtimes loaded) is hurting performance, and engineers are demotivated because the coexistence layer is fragile and slows down day-to-day feature work in both frameworks.

Question: After three years of a stalled hybrid migration, how do you get it unstuck — do you push through with the hybrid approach, or change strategy entirely?

Answer: A three-year stalled migration is a strong signal that the current strategy itself is the problem, not just execution speed — I'd stop and re-diagnose rather than "push harder" on the same approach.

1. Diagnose why it stalled — usually one of: (a) migration work has no dedicated capacity and is perpetually deprioritized against feature work, (b) the hardest 20% of routes (deeply stateful, tightly coupled to AngularJS services) were left for last and are now blocking everything, or (c) the hybrid coexistence tax itself (bundle size, `$digest` /zone.js interop overhead, dual mental model for engineers) is so costly that teams unconsciously avoid touching migrated code, slowing everything down.

2. If it's (a) — capacity/prioritization — the fix is organizational: carve out explicit migration capacity (e.g., 20% dedicated time per team, or a dedicated rotation) with visible executive sponsorship, since "migrate opportunistically alongside feature work" demonstrably hasn't worked for three years.

3. If it's (b) — hardest routes last — reconsider route selection strategy entirely: prioritize by *coupling*, not by ease. Routes tightly coupled to shared AngularJS services should often be tackled *earlier*, specifically because they block the most other things, even though they're harder — "easy wins first" optimizes for a nice-looking burndown chart, not for unblocking the migration's critical path.

4. If it's (c) — hybrid tax is too costly — consider whether a full rewrite of the remaining 60% (rather than continued incremental hybrid migration) is now the better economic choice. Three more years of hybrid overhead across all users, versus a focused rewrite effort, is a real cost comparison worth making explicit with data (bundle size delta, performance metrics, engineer-days spent working around the interop layer) rather than assuming "keep going incrementally" is always right just because it's what's already underway — sunk cost is a trap here.

```
// A frequent hybrid-migration blocker: shared AngularJS services accessed via `$injector`  
// need explicit upgrade adapters, and every new dependency between the two worlds  
// adds a fragile interop point – audit how many of these exist; a high count itself  
// signals the coexistence layer is the actual bottleneck, not individual route complexity  
angular.module('app').factory('legacyUserService', downgradeInjectable(UserService));
```

5. Whatever the diagnosis, set a hard, board-visible deadline for full AngularJS removal, since open-ended migrations without a forcing function tend to never finish — three years without one is itself the strongest evidence for this. Tie it to a concrete forcing event if possible (e.g., an upcoming security/compliance requirement, or a hard AngularJS end-of-life/support cutoff) to create organizational urgency that pure engineering advocacy hasn't been able to generate on its own.

6. Communicate re-planning honestly upward: presenting "here's why the current approach stalled, here's the data-backed alternative, here's the new deadline and required capacity" to leadership is a stronger position than quietly continuing a strategy that's demonstrably not working, even though admitting three years of hybrid investment needs a course correction is a hard conversation.

Interviewer intent: Tests long-horizon migration judgment — recognizing when to abandon a stalled strategy rather than "just execute harder," diagnosing root cause (capacity vs. sequencing vs. inherent hybrid cost) before recommitting, and creating organizational forcing functions for indefinite migrations.

Scenario 22: Observability Gaps — Frontend Errors Invisible Until a Customer Complains

Situation: Your enterprise app serves large corporate clients, several with strict internal browser/proxy configurations. Production errors are currently only discovered when a customer's admin emails support, and by the time engineering investigates, the browser session and any useful console state are long gone. Leadership wants to know why "we don't see problems before our customers do."

Question: Design an end-to-end frontend observability strategy for an enterprise Angular application, addressing both error visibility and the constraints of an enterprise-controlled environment (proxies, ad-blockers, CSP policies).

Answer: Enterprise frontend observability has real constraints that consumer-app tooling assumptions don't account for: corporate proxies may block third-party error-reporting domains, ad-blockers/security software may block known telemetry endpoints, and strict CSP policies (often set by the *client's* IT department, not yours) may prevent script/connect access to external monitoring vendors entirely. So the design needs to route around those constraints, not assume a SaaS error-tracking script "just works."

1. Global error handler capturing everything, with enough context to actually debug it (not just a stack trace):

```
@Injectable()
export class ObservableErrorHandler implements ErrorHandler {
  private telemetry = inject(TelemetryService);
  private router = inject(Router);

  handleError(error: unknown): void {
    this.telemetry.report({
      message: error instanceof Error ? error.message : String(error),
      stack: error instanceof Error ? error.stack : undefined,
      url: this.router.url,
      tenantId: this.telemetry.currentTenantId(),
      appVersion: environment.appVersion,
      timestamp: Date.now(),
      // Breadcrumbs: last N user actions/route changes, since the console is long gone by
      // the time you investigate
      breadcrumbs: this.telemetry.getRecentBreadcrumbs(),
    });
    console.error(error); // still log locally for anyone with devtools open
  }
}
```

2. Route error reporting through your *own* first-party backend endpoint, not only a third-party SaaS domain, since enterprise proxies/CSP are far more likely to allow a request to `api.yourcompany.com/telemetry` (a domain the client has already allow-listed for the app to function at all) than to an error-tracking vendor's domain the client's IT team has never seen and may block by default.

```
// First-party proxy endpoint – your backend forwards to Sentry/Datadog/etc. internally if
// desired,
// but the browser only ever talks to a domain the enterprise client has already allow-
// listed
this.http.post('/api/telemetry/errors', payload).subscribe();
```

3. Breadcrumb trail via a lightweight, in-memory ring buffer (route changes, key user actions, last few HTTP calls with status codes) attached to every reported error, since "what did the user do right before this broke" is usually the actual missing piece when investigating without a live session.

4. Respect CSP explicitly, don't fight it: publish a documented CSP snippet enterprise clients' IT teams can add (`connect-src 'self' https://api.yourcompany.com`) as part of your onboarding/deployment docs, so telemetry isn't silently dropped in the client's stricter environments — this turns "we don't see problems before customers do" partly into a *documentation* gap (clients' proxies blocking telemetry silently) as much as a tooling gap.

5. Session replay/context capture, privacy-conscious for enterprise clients: rather than full session replay (often against enterprise clients' data policies and a compliance risk with sensitive corporate data on screen), capture structured state snapshots (current route, feature flags active, last API responses' status codes, form validation state) which give most of the debugging value without capturing PII-risk visual data.

6. Proactive synthetic monitoring (scripted Playwright runs against production on a schedule, hitting critical flows) as a complement to passive error reporting, specifically so critical-path breakage is caught by *your* monitoring within minutes, independent of whether real-user telemetry made it through a given client's proxy at all.

7. Alert on error-rate anomalies per tenant, not just globally — a global error-rate dashboard can look healthy while one specific large enterprise client is experiencing elevated errors (e.g., due to their specific browser lockdown policy or proxy config), and per-tenant breakdown is what actually surfaces "why is this one client having a bad time" before their admin emails support.

Interviewer intent: Tests awareness that enterprise frontend observability has different constraints than typical SaaS/consumer telemetry (proxies, CSP, ad-blockers, privacy policies) and requires first-party routing plus per-tenant granularity, not just "add Sentry."

Quick Revision Cheat Sheet

- **Diagnose before architecting:** slow builds, forked components, and stalled migrations usually have a root cause (coupling, process bottleneck, capacity) that a trendy pattern (micro-frontends, full rewrite) won't fix on its own — measure first.
- **Breaking changes in shared libraries are additive-first:** new API alongside old, deprecation warnings, an automated codemod/schematic, then removal in a well-announced major version — never a silent big-bang break.
- **Design-system governance is a process SLA problem as much as a code problem:** slow review cycles cause forking; fix both the contribution process and make components extensible via DI/content-projection.
- **NgModule → standalone migration works because of bidirectional interop:** migrate leaf components first, routed/lazy modules last, use `ng generate @angular/core:standalone`, and ban new NgModules going forward.
- **Onboarding friction is a legitimate signal:** invest in a golden-path example and an architecture decision map; don't force-migrate mismatched state-management patterns without a concrete pain point.
- **Global cross-cutting code (interceptors, guards) needs elevated review and explicit opt-in mechanisms** (`HttpContextToken`) rather than implicit global behavior that silently affects every team.
- **Enforce bundle budgets with `maximumError`, not just warnings,** and watch for cross-chunk duplication (a build-config/shared-entry-point problem, not a per-team lazy-loading discipline problem).
- **Sequence org-wide upgrades by risk** (canary apps first, revenue-critical last) and give teams a playbook/codemod, not just a mandate and a deadline.
- **Zoneless migration's real risk is "invisible" state mutation** (callbacks from jQuery/third-party SDKs that zone.js used to silently paper over — audit and convert those to signals explicitly).
- **Compile-time types don't catch runtime API drift:** generate types from OpenAPI specs, validate at the HTTP boundary (zod), and use consumer-driven contract tests to shift breakage detection to the team that caused it.
- **Feature flags need a lifecycle** (release/ops/experiment classification, expiry dates, owners) and centralized interaction-resolution logic, not ad-hoc scattered conditionals across components.
- **Make correct error handling the default** via a global interceptor + `ErrorHandler`, not something

every developer must remember per HTTP call.

- **Pick state management by requirements fit** (undo/redo and time-travel debugging favor NgRx; simple CRUD favors signals/plain services), not by existing-codebase inertia in either direction.
- **A slow E2E gate is usually an inverted test pyramid:** parallelize/shard, quarantine flaky tests with owner SLAs, push logic coverage down to unit/component tests, and reserve E2E for the critical-path few.
- **Multi-tenant theming/entitlements should be data-driven** (CSS custom properties, capability tokens/directives/guards) so onboarding a new client is a config change, not a code change.
- **Cross-team technical adoption needs demonstrated value and an interop bridge**, not a top-down mandate — especially against a deadline the resisting teams are worried about.
- **Nx build-time scaling needs remote/distributed caching and execution, plus fixing overly-broad shared libraries** whose changes cascade to far more dependents than necessary.
- **A11y and technical-debt-vs-deadline conflicts are best resolved by translating cost into business terms** and offering scoped middle options, with the final tradeoff made an explicit, shared, written decision.
- **Enterprise frontend observability must route through first-party endpoints** and account for client-side proxies/CSP/ad-blockers silently dropping third-party telemetry, with per-tenant error-rate visibility.
- **Long-stalled migrations (multi-year hybrid coexistence) call for re-diagnosis, not more of the same effort:** identify whether capacity, sequencing, or inherent coexistence cost is the blocker, and attach a hard deadline with organizational forcing functions.

Created By - Durgesh Singh